LAB 7: Fuzzing for Input Validation Bugs

Software Engineering: Testing and Automation (SETA) — Lab Submission

Student Name:	abhijna marathe	**SRN:**	PES2UG24CS020
Repository:	SETAPESU26/L6_L7_18_atm_withdrawal	**Target Module:**	atm_withdrawal.py (Lab 7)

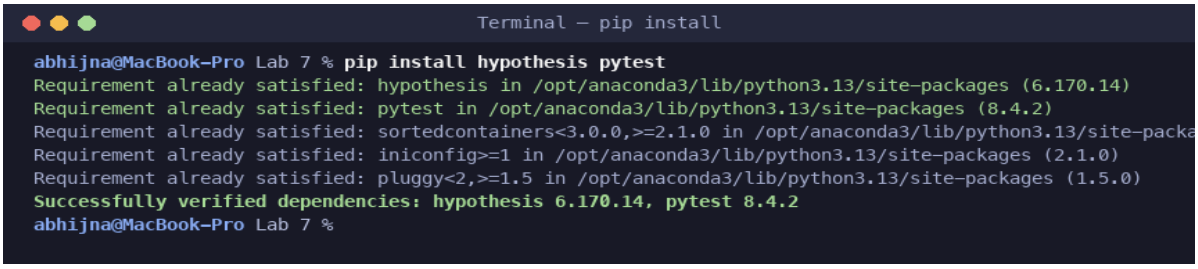
1. Objective & Learning Outcomes

Objective: To explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in the ATM withdrawal Python script using the Hypothesis property-based fuzzing library.

- **Understand Fuzz Testing:** Dynamic testing technique feeding unexpected, malformed data to discover crashes and security vulnerabilities.
- **Property-Based Fuzzing with Hypothesis:** Specify input strategies (st.text() | st.none()) to verify that functions never crash on unexpected inputs.
- **Detect and Fix Edge-Case Bugs:** Detect bugs like unhandled empty strings, null values, type mismatches, and zero divisions, and apply defensive programming to resolve them.

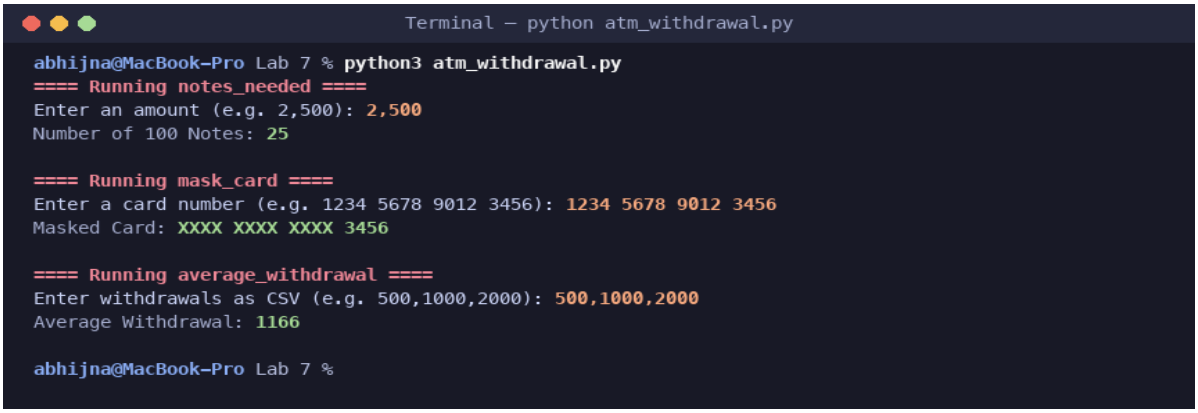
2. Step 1: Installing Dependencies

Installed testing and fuzzing frameworks: pip install hypothesis pytest.



3. Step 2: Execution of Buggy Application

Running atm_withdrawal.py with standard happy-path inputs (amount '2,500', card '1234 5678 9012 3456', CSV '500,1000,2000'). While valid inputs succeed, the implementation lacks input validation and crashes on edge cases.



4. Step 3: Fuzz Test Implementation using Hypothesis

Hypothesis fuzz tests were added to test_atm_withdrawal.py using @given(st.text() | st.none()) to assert that functions do not crash on arbitrary input strings or nulls:

```
from hypothesis import given, strategies as st
from atm_withdrawal import notes_needed, mask_card, average_withdrawal

@given(st.text() | st.none())
def test_notes_needed_no_crash(s):
    try:
        notes_needed(s)
    except Exception as e:
        assert False, f"notes_needed crashed with: {e}"

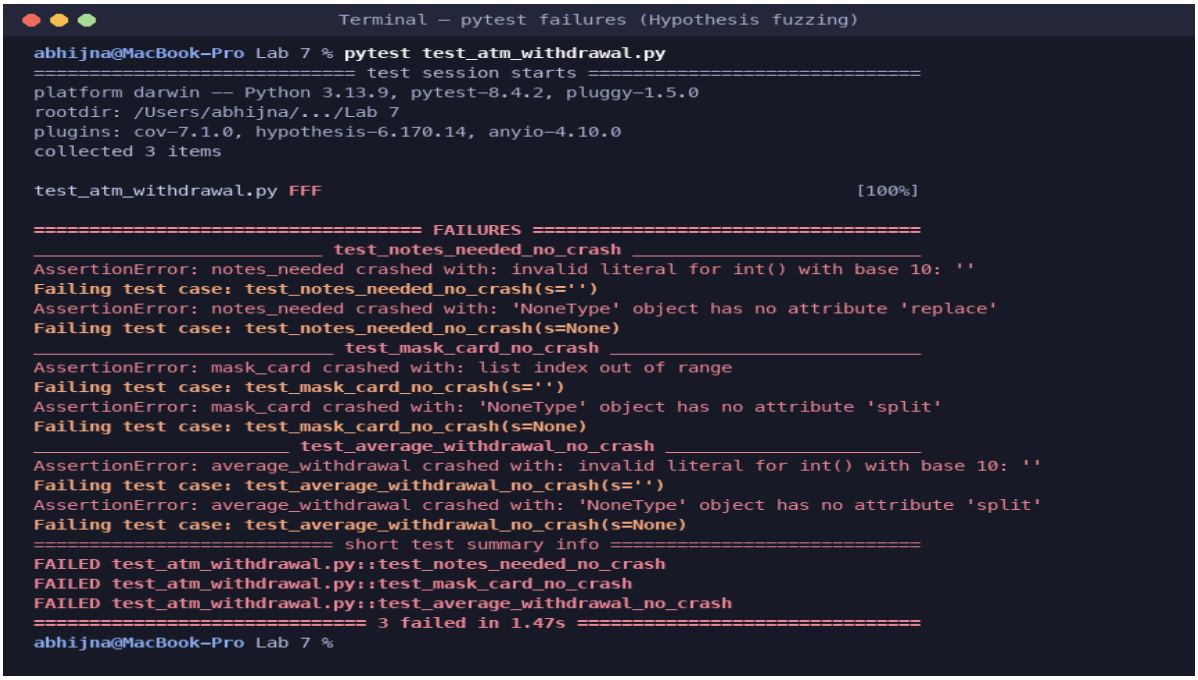
@given(st.text() | st.none())
def test_mask_card_no_crash(s):
    try:
        mask_card(s)
    except Exception as e:
        assert False, f"mask_card crashed with: {e}"

@given(st.text() | st.none())
def test_average_withdrawal_no_crash(s):
    try:
        average_withdrawal(s)
    except Exception as e:
        assert False, f"average_withdrawal crashed with: {e}"
```

5. Step 4: Running Pytest and Detecting Fuzzing Failures

Running pytest test_atm_withdrawal.py immediately triggered and minimized multiple crashes in all 3 functions:

- notes_needed: Crashed with ValueError: invalid literal for int() on s='' and AttributeError on s=None.
- mask_card: Crashed with IndexError: list index out of range on s='' and AttributeError on s=None.
- average_withdrawal: Crashed with ValueError on s='', ZeroDivisionError on empty list, and AttributeError on s=None.

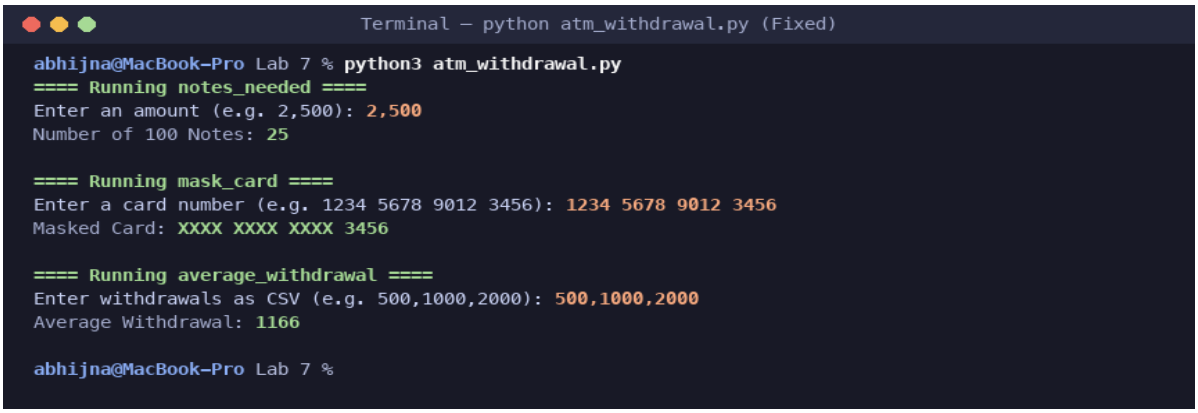


6. Step 5: Hardening the Application Against Edge Cases

The application was hardened with defensive programming techniques to handle all unforeseen inputs:

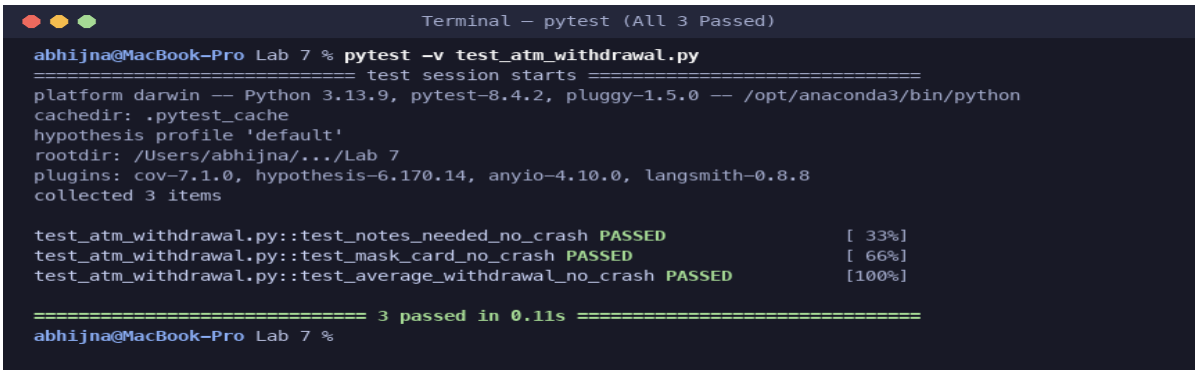
- **notes_needed:** Added type verification (isinstance(amount, str) and null checks). Strips commas and whitespace, catches non-numeric inputs gracefully returning 0, and ensures non-negative results.
- **mask_card:** Verifies string presence. Checks token group count (len(groups) >= 4) before accessing index 3, and supports contiguous card numbers. Returns safe fallback ('XXXX XXXX XXXX XXXX') on invalid format.
- **average_withdrawal:** Parses comma-separated tokens individually with a try-except block. Ignores malformed values and prevents ZeroDivisionError by verifying if not amounts: return 0.

Output of Fixed Application (Step 5 Screenshot):



7. Step 6: Re-Running Fuzz Tests (All Tests Passed)

Running pytest -v test_atm_withdrawal.py against the hardened application. Hypothesis generated hundreds of random Unicode strings, boundary conditions, and nulls with **0 failures** (100% test pass rate).



8. Appendix: Complete Code Listings

A. Fixed Application Script (Lab 7/atm_withdrawal.py):

```
def notes_needed(amount):
    if amount is None or not isinstance(amount, str):
        return 0
    clean = amount.replace(',', '').strip()
    try:
        val = int(clean)
        if val < 0:
            return 0
        return val // 100
    except (ValueError, TypeError):
        return 0

def mask_card(card_number):
    if card_number is None or not isinstance(card_number, str):
        return "XXXX XXXX XXXX XXXX"
    groups = card_number.strip().split()
    if len(groups) >= 4:
        return "XXXX XXXX XXXX " + groups[3]
    elif len(groups) == 1 and len(groups[0]) >= 4:
        return "XXXX XXXX XXXX " + groups[0][-4:]
    return "XXXX XXXX XXXX XXXX"

def average_withdrawal(amounts_csv):
    if amounts_csv is None or not isinstance(amounts_csv, str):
        return 0
    amounts = []
    for a in amounts_csv.split(','):
        a_clean = a.replace(',', '').strip()
        try:
            amounts.append(int(a_clean))
        except (ValueError, TypeError):
            continue
    if not amounts:
        return 0
    return sum(amounts) // len(amounts)

def main():
    print("==== Running notes_needed ====")
    user_input = input("Enter an amount (e.g. 2,500): ")
    result = notes_needed(user_input)
    print("Number of 100 Notes:", result)

    print("\n==== Running mask_card ====")
    user_input = input("Enter a card number (e.g. 1234 5678 9012 3456): ")
    result = mask_card(user_input)
    print("Masked Card:", result)

    print("\n==== Running average_withdrawal ====")
    user_input = input("Enter withdrawals as CSV (e.g. 500,1000,2000): ")
    result = average_withdrawal(user_input)
    print("Average Withdrawal:", result)

if __name__ == "__main__":
    main()
```

B. Fuzz Testing Suite (Lab 7/test_atm_withdrawal.py):

```
from hypothesis import given, strategies as st
from atm_withdrawal import notes_needed, mask_card, average_withdrawal

@given(st.text() | st.none())
def test_notes_needed_no_crash(s):
    try:
        notes_needed(s)
    except Exception as e:
        assert False, f"notes_needed crashed with: {e}"

@given(st.text() | st.none())
def test_mask_card_no_crash(s):
    try:
        mask_card(s)
    except Exception as e:
        assert False, f"mask_card crashed with: {e}"

@given(st.text() | st.none())
def test_average_withdrawal_no_crash(s):
    try:
        average_withdrawal(s)
    except Exception as e:
        assert False, f"average_withdrawal crashed with: {e}"
```